

So, you want to build your first algorithmic trading model. As daunting as it may seem, using a powerful Python library called VectorBT can drastically simplify the process and make it a breeze. I personally love VectorBT as it is free and lets us build trading and backtesting programs in shockingly few lines of code. In this article we'll build a model that uses only moving average indicators, and in the next we will explore other strategies.

Before the meat of the article, a few prerequisites:

```
In [1]: pip install vectorbt
```

```
Requirement already satisfied: vectorbt in ./anaconda3/lib/python3.11/site-packages (0.26.1)
Requirement already satisfied: numpy>=1.16.5 in ./anaconda3/lib/python3.11/site-packages (from vectorbt) (1.24.4)
Requirement already satisfied: pandas in ./anaconda3/lib/python3.11/site-packages (from vectorbt) (1.5.3)
Requirement already satisfied: scipy in ./anaconda3/lib/python3.11/site-packages (from vectorbt) (1.11.1)
Requirement already satisfied: matplotlib in ./anaconda3/lib/python3.11/site-packages (from vectorbt) (3.7.2)
Requirement already satisfied: plotly>=4.12.0 in ./anaconda3/lib/python3.11/site-packages (from vectorbt) (5.19.0)
Requirement already satisfied: ipywidgets>=7.0.0 in ./anaconda3/lib/python3.11/site-packages (from vectorbt) (8.1.2)
Requirement already satisfied: dill in ./anaconda3/lib/python3.11/site-packages (from vectorbt) (0.3.6)
Requirement already satisfied: tqdm in ./anaconda3/lib/python3.11/site-packages (from vectorbt) (4.65.0)
Requirement already satisfied: dateparser in ./anaconda3/lib/python3.11/site-packages (from vectorbt) (1.2.0)
Requirement already satisfied: imageio in ./anaconda3/lib/python3.11/site-packages (from vectorbt) (2.31.1)
Requirement already satisfied: scikit-learn in ./anaconda3/lib/python3.11/site-packages (from vectorbt) (1.4.1.post1)
Requirement already satisfied: schedule in ./anaconda3/lib/python3.11/site-packages (from vectorbt) (1.2.1)
Requirement already satisfied: requests in ./anaconda3/lib/python3.11/site-packages (from vectorbt) (2.31.0)
Requirement already satisfied: pytz in ./anaconda3/lib/python3.11/site-packages (from vectorbt) (2023.3.post1)
Requirement already satisfied: mpy-extensions in ./anaconda3/lib/python3.11/site-packages (from vectorbt) (1.0.0)
Requirement already satisfied: numba>=0.57.0 in ./anaconda3/lib/python3.11/site-packages (from vectorbt) (0.57.0)
Requirement already satisfied: comm>=0.1.3 in ./anaconda3/lib/python3.11/site-packages (from ipywidgets>=7.0.0->vectorbt) (0.2.1)
Requirement already satisfied: ipython>=6.1.0 in ./anaconda3/lib/python3.11/site-packages (from ipywidgets>=7.0.0->vectorbt) (8.15.0)
Requirement already satisfied: traitlets>=4.3.1 in ./anaconda3/lib/python3.11/site-packages (from ipywidgets>=7.0.0->vectorbt) (5.7.1)
Requirement already satisfied: widgetsnbextension~=4.0.10 in ./anaconda3/lib/python3.11/site-packages (from ipywidgets>=7.0.0->vectorbt) (4.0.10)
Requirement already satisfied: jupyterlab-widgets~=3.0.10 in ./anaconda3/lib/python3.11/site-packages (from ipywidgets>=7.0.0->vectorbt) (3.0.10)
Requirement already satisfied: llvmlite<0.41,>=0.40.0dev0 in ./anaconda3/lib/python3.11/site-packages (from numba>=0.57.0->vectorbt) (0.40.0)
Requirement already satisfied: tenacity>=6.2.0 in ./anaconda3/lib/python3.11/site-packages (from plotly>=4.12.0->vectorbt) (8.2.2)
Requirement already satisfied: packaging in ./anaconda3/lib/python3.11/site-packages (from plotly>=4.12.0->vectorbt) (23.2)
Requirement already satisfied: python-dateutil in ./anaconda3/lib/python3.11/site-packages (from dateparser->vectorbt) (2.8.2)
Requirement already satisfied: regex!=2019.02.19,!=2021.8.27 in ./anaconda3/lib/python3.11/site-packages (from dateparser->vectorbt) (2022.7.9)
Requirement already satisfied: tzlocal in ./anaconda3/lib/python3.11/site-packages (from dateparser->vectorbt) (5.2)
Requirement already satisfied: pillow>=8.3.2 in ./anaconda3/lib/python3.11/site-packages (from imageio->vectorbt) (9.4.0)
Requirement already satisfied: contourpy>=1.0.1 in ./anaconda3/lib/python3.11/site-packages (from matplotlib->vectorbt) (1.0.5)
Requirement already satisfied: cycler>=0.10 in ./anaconda3/lib/python3.11/site-packages (from matplotlib->vectorbt) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in ./anaconda3/lib/python3.11/site-packages (from matplotlib->vectorbt) (4.25.0)
Requirement already satisfied: kiwisolver>=1.0.1 in ./anaconda3/lib/python3.11/site-packages (from matplotlib->vectorbt) (1.4.4)
Requirement already satisfied: pyparsing<3.1,>=2.3.1 in ./anaconda3/lib/python3.11/site-packages (from matplotlib->vectorbt) (3.0.9)
Requirement already satisfied: charset-normalizer<4,>=2 in ./anaconda3/lib/python3.11/site-packages (from requests->vectorbt) (2.0.4)
Requirement already satisfied: idna<4,>=2.5 in ./anaconda3/lib/python3.11/site-packages (from requests->vectorbt) (3.4)
Requirement already satisfied: urllib3<3,>=1.21.1 in ./anaconda3/lib/python3.11/site-packages (from requests->vectorbt) (1.26.16)
Requirement already satisfied: certifi>=2017.4.17 in ./anaconda3/lib/python3.11/site-packages (from requests->vectorbt) (2023.7.22)
Requirement already satisfied: joblib>=1.2.0 in ./anaconda3/lib/python3.11/site-packages (from scikit-learn->vectorbt) (1.4.2)
Requirement already satisfied: threadpoolctl>=2.0.0 in ./anaconda3/lib/python3.11/site-packages (from scikit-learn->vectorbt) (3.3.0)
Requirement already satisfied: backcall in ./anaconda3/lib/python3.11/site-packages (from ipython>=6.1.0->ipywidgets>=7.0.0->vectorbt) (0.2.0)
Requirement already satisfied: decorator in ./anaconda3/lib/python3.11/site-packages (from ipython>=6.1.0->ipywidgets>=7.0.0->vectorbt) (5.1.1)
Requirement already satisfied: jedi>=0.16 in ./anaconda3/lib/python3.11/site-packages (from ipython>=6.1.0->ipywidgets>=7.0.0->vectorbt) (0.18.1)
Requirement already satisfied: matplotlib-inline in ./anaconda3/lib/python3.11/site-packages (from ipython>=6.1.0->ipywidgets>=7.0.0->vectorbt) (0.1.6)
```

```
Requirement already satisfied: pickleshare in ./anaconda3/lib/python3.11/site-packages (from ipython>=6.1.0->ipywidgets>=7.0.0->vectorbt) (0.7.5)
Requirement already satisfied: prompt-toolkit!=3.0.37,<3.1.0,>=3.0.30 in ./anaconda3/lib/python3.11/site-packages (from ipython>=6.1.0->ipywidgets>=7.0.0->vectorbt) (3.0.36)
Requirement already satisfied: pygments>=2.4.0 in ./anaconda3/lib/python3.11/site-packages (from ipython>=6.1.0->ipywidgets>=7.0.0->vectorbt) (2.15.1)
Requirement already satisfied: stack-data in ./anaconda3/lib/python3.11/site-packages (from ipython>=6.1.0->ipywidgets>=7.0.0->vectorbt) (0.2.0)
Requirement already satisfied: pexpect>4.3 in ./anaconda3/lib/python3.11/site-packages (from ipython>=6.1.0->ipywidgets>=7.0.0->vectorbt) (4.8.0)
Requirement already satisfied: appnope in ./anaconda3/lib/python3.11/site-packages (from ipython>=6.1.0->ipywidgets>=7.0.0->vectorbt) (0.1.2)
Requirement already satisfied: six>=1.5 in ./anaconda3/lib/python3.11/site-packages (from python-dateutil->dateparser->vectorbt) (1.16.0)
Requirement already satisfied: parso<0.9.0,>=0.8.0 in ./anaconda3/lib/python3.11/site-packages (from jedi>=0.16->ipython>=6.1.0->ipywidgets>=7.0.0->vectorbt) (0.8.3)
Requirement already satisfied: ptyprocess>=0.5 in ./anaconda3/lib/python3.11/site-packages (from pexpect>4.3->ipython>=6.1.0->ipywidgets>=7.0.0->vectorbt) (0.7.0)
Requirement already satisfied: wcwidth in ./anaconda3/lib/python3.11/site-packages (from prompt-toolkit!=3.0.37,<3.1.0,>=3.0.30->ipython>=6.1.0->ipywidgets>=7.0.0->vectorbt) (0.2.5)
Requirement already satisfied: executing in ./anaconda3/lib/python3.11/site-packages (from stack-data->ipython>=6.1.0->ipywidgets>=7.0.0->vectorbt) (0.8.3)
Requirement already satisfied: asttokens in ./anaconda3/lib/python3.11/site-packages (from stack-data->ipython>=6.1.0->ipywidgets>=7.0.0->vectorbt) (2.0.5)
Requirement already satisfied: pure-eval in ./anaconda3/lib/python3.11/site-packages (from stack-data->ipython>=6.1.0->ipywidgets>=7.0.0->vectorbt) (0.2.2)
Note: you may need to restart the kernel to use updated packages.
```

```
In [2]: pip install datetime
```

```
Requirement already satisfied: datetime in ./anaconda3/lib/python3.11/site-packages (5.5)
Requirement already satisfied: zope.interface in ./anaconda3/lib/python3.11/site-packages (from datetime) (5.4.0)
Requirement already satisfied: pytz in ./anaconda3/lib/python3.11/site-packages (from datetime) (2023.3.post1)
Requirement already satisfied: setuptools in ./anaconda3/lib/python3.11/site-packages (from zope.interface->datetime) (70.0.0)
Note: you may need to restart the kernel to use updated packages.
```

Theory:

The strategy we'll build in this article is simple, and notably not the most effective in the real world, but is easy and a good example to learn how to build a backtesting program. The strategy works like this: you record two moving averages as the price of whatever you're trading changes. One of these moving averages is a "short" moving average and the other is a "long" moving average— in my code I use "fast" and "slow" to represent these two moving averages. The strategy is based on the idea that a security can have momentum with buying and selling, so when the short moving average crosses above the long moving average, the security has positive momentum and will likely trend upwards, so you buy (This is called the "Golden Cross"). Contrarily when the short moving average crosses below the long moving average the price has negative momentum, so you sell (This is called the "Death Cross"). The fact that this depends purely on a single indicator can cause it to send false signals, so once again, it is not a great strategy to use on its own, but is an excellent example to learn with and can be more effective when paired with other indicators.

Code:

First, import the two libraries we'll be using: VectorBT and datetime.

```
In [3]: import vectorbt as vbt
import datetime as dt
```

```
/Users/abdelkarimabdallah/anaconda3/lib/python3.11/site-packages/yfinance/base.py:48: FutureWarning: The default dtype for empty Series will be 'object' instead of 'float64' in a future version. Specify a dtype explicitly to silence this warning.
    _empty_series = pd.Series()
```

Next, we want to set the backtesting window for the amount of time we'll be testing over. We need the datetime library for this, and use the methods now() and timedelta() for this. The first line gets the current date/time, and the second initializes the startDate variable and sets it equal to 730 days (2 years) prior to today by subtracting 730 days from the current date (represented by the "rn" variable) using the timedelta() method.

```
In [4]: rn = dt.datetime.now()
startDate = rn - dt.timedelta(days=730)
```

Next we import the data using VectorBT. The first argument in the download() method is the symbol of the security ("SPY"), the second is the interval you want the data for ("1d"), and the last is the start date of the data (start=startDate). There are other arguments you can pass in the download() method but this is all we need for this program. Finally we use the get() method to choose what data we actually want to use from what we just downloaded. We pass the 'Close' argument to use just the closing price data.

```
In [6]: symbol = vbt.YFData.download('spy', interval='1d', start=startDate).get('Close')
```

In the more data science oriented part of the code, we use VectorBT to analyze the closing price data and generate the moving average data from that. We use two different variables, "fast_ma" and "slow_ma" to do this. the run() method is a VectorBT method that will apply whatever indicator precedes it ('MA' in this case) to the data which is passed in the first argument. When you use the run() method with the moving average indicator as we do here, the only other argument we must pass is the window. In my example we use 50 days (fast) and 100 days (slow).

```
In [7]: fast_ma = vbt.MA.run(symbol, 50)
slow_ma = vbt.MA.run(symbol, 100)
```

Beginning the trading specific section of the code, we define the entry and exit conditions. To define these conditions you use one of the data sets you created before (either fast_ma or slow_ma) then you use a VectorBT method to compare it to another. In this case we define an entry as when the fast_ma crosses above the slow_ma, and an exit is the opposite.

```
In [8]: entries = fast_ma.ma_crossed_above(slow_ma)
exits = fast_ma.ma_crossed_below(slow_ma)
```

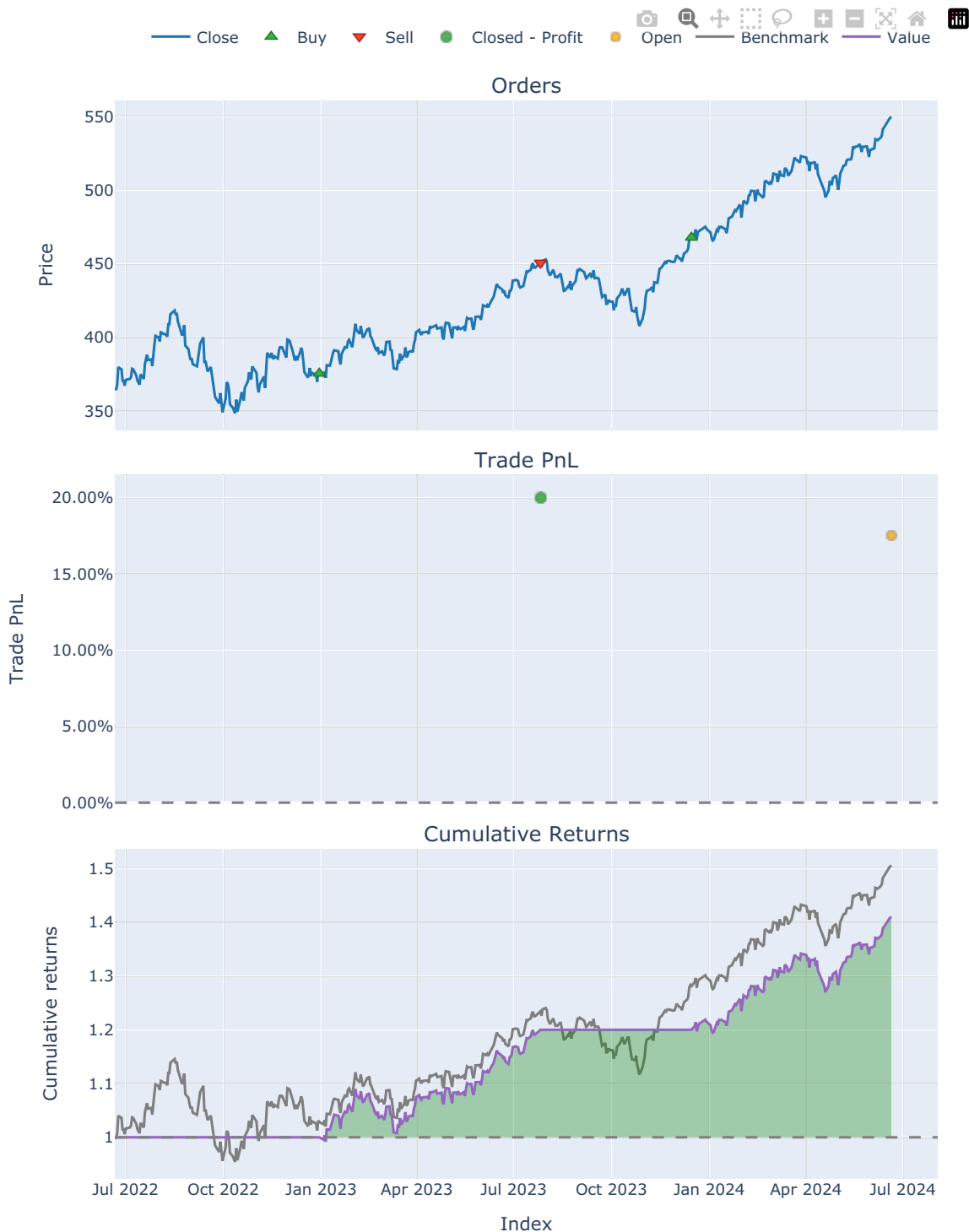
Finally, we use the VectorBT "Portfolio" class to put all of our previous work together, and perform the backtest of our strategy. From the Portfolio class we use the method from_signals to add the signals we've created (entries and exits) to the strategy. After the first three arguments in the method (symbol, entries and exits) the remaining arguments are "init_cash" which sets the starting value/buying power of our portfolio, "freq" which states the frequency which the program can trade on, "sl_stop" which allows us to set a stop loss parameter, and "tp_stop" which allows us to set a take-profit parameter.

```
In [9]: pf = vbt.Portfolio.from_signals(symbol, entries, exits, init_cash=100, freq='1d', sl_stop=0.05, tp_stop=0.2)
```

Finally we use a couple more lines of code to display our results. The final line, which creates the graph and displays it, uses VectorBT.

```
In [10]: print(pf.total_profit())
print(pf.stats())
pf.plot().show()
```

```
41.05371689383354
Start                2022-06-21 04:00:00+00:00
End                  2024-06-20 04:00:00+00:00
Period               503 days 00:00:00
Start Value          100.0
End Value             141.053717
Total Return [%]      41.053717
Benchmark Return [%]  50.612534
Max Gross Exposure [%] 100.0
Total Fees Paid       0.0
Max Drawdown [%]      7.538746
Max Drawdown Duration 58 days 00:00:00
Total Trades          2
Total Closed Trades   1
Total Open Trades     1
Open Trade PnL        21.045628
Win Rate [%]          100.0
Best Trade [%]        20.008089
Worst Trade [%]       20.008089
Avg Winning Trade [%] 20.008089
Avg Losing Trade [%]   NaN
Avg Winning Trade Duration 141 days 00:00:00
Avg Losing Trade Duration NaT
Profit Factor         inf
Expectancy            20.008089
Sharpe Ratio          2.321716
Calmar Ratio          3.760746
Omega Ratio           1.544155
Sortino Ratio         3.755657
dtype: object
```



For ease of copying and pasting to test it out, here is the full code:

```
In [11]: import vectorbt as vbt
import datetime as dt

rn = dt.datetime.now()
startDate = rn - dt.timedelta(days=730)

symbol = vbt.YFData.download('SPY', interval='1d', start=startDate).get('Close')

fast_ma = vbt.MA.run(symbol, 50)
slow_ma = vbt.MA.run(symbol, 100)

entries = fast_ma.ma_crossed_above(slow_ma)
exits = fast_ma.ma_crossed_below(slow_ma)

pf = vbt.Portfolio.from_signals(symbol, entries, exits, init_cash=100, freq='1d', sl_stop=0.05, tp_stop=0.2)
print(pf.total_profit())
print(pf.stats())
pf.plot().show()
```

41.026273269422845	
Start	2022-06-21 04:00:00+00:00
End	2024-06-20 04:00:00+00:00
Period	503 days 00:00:00
Start Value	100.0
End Value	141.026273
Total Return [%]	41.026273
Benchmark Return [%]	50.583218
Max Gross Exposure [%]	100.0
Total Fees Paid	0.0
Max Drawdown [%]	7.538754
Max Drawdown Duration	58 days 00:00:00
Total Trades	2
Total Closed Trades	1
Total Open Trades	1
Open Trade PnL	21.018184
Win Rate [%]	100.0
Best Trade [%]	20.008089
Worst Trade [%]	20.008089
Avg Winning Trade [%]	20.008089
Avg Losing Trade [%]	NaN
Avg Winning Trade Duration	141 days 00:00:00
Avg Losing Trade Duration	NaT
Profit Factor	inf
Expectancy	20.008089
Sharpe Ratio	2.320479
Calmar Ratio	3.758339
Omega Ratio	1.543854
Sortino Ratio	3.753576
dtype: object	

In []: `## I hope you enjoyed the article`